

Lost Update

Lost Update occurs when **two transactions update the same data item at the same time**, and the update made by one transaction is **overwritten by the other**.

Example:

Two transactions read the same account balance and update it.
The final update of one transaction **removes the effect of the other**.

Uncommitted Dependency (Dirty Read)

Uncommitted Dependency happens when **one transaction reads data written by another transaction that has not yet been committed**.

Example:

Transaction T1 updates a balance but does not commit.
Transaction T2 reads this updated value.
Later, T1 is aborted, so T2 has read **incorrect data**.

Inconsistent Analysis

Inconsistent Analysis occurs when a transaction **reads several data items for analysis**, while another transaction **changes some of those data items during the analysis**.

Example:

While calculating the total balance of all accounts, another transaction updates some balances, resulting in a **wrong total**.

1 Atomicity (সব নয় তো কিছুই নয়)

একটা transaction-এর সব কাজ একসাথে ঠিকমতো হলে তবেই save হবে।

একটা কাজও fail হলে সব কাজ বাতিল (rollback) হবে।

👉 উদাহরণ: টাকা পাঠাতে গিয়ে যদি টাকা কাটা হয় কিন্তু জমা না হয় → transaction fail।

2 Consistency (ডাটা সবসময় ঠিক থাকবে)

Transaction শেষ হলে ডাটাবেজ **নিয়ম মেনে সঠিক অবস্থায়** থাকবে।
কোনো rule বা constraint ভাঙবে না।

👉 উদাহরণ: Account balance কখনো negative হওয়া যাবে না।

3 Isolation (একটা কাজ আরেকটার উপর প্রভাব ফেলবে না)

একই সময় অনেক transaction চললেও,
প্রতিটা transaction আলাদা আলাদা মনে হবে।

👉 উদাহরণ: দুইজন একসাথে টাকা তুললেও হিসাব গুলমাল হবে না।

4 Durability (একবার save হলে আর হারাবে না)

Transaction সফল হলে ডাটা **স্থায়ীভাবে save** থাকবে,
বিদ্যুৎ চলে গেলেও বা system crash হলেও।

👉 উদাহরণ: “Transaction Successful” দেখালে টাকা হারাবে না।

Transitive Dependency

A **transitive dependency** happens when a **non-key attribute depends on another non-key attribute** instead of directly depending on the primary key.

Example:

If $A \rightarrow B$ and $B \rightarrow C$, then $A \rightarrow C$ is a transitive dependency.

Insertion Anomaly

An **insertion anomaly** occurs when you **cannot insert new data** into a table because **some other required data is missing**.

Example:

You can't add a new department unless at least one employee is assigned to it.

Explain what you understand by serializability of schedule

Serializability of a Schedule means a schedule (order of operations of transactions) is correct if its result is the same as some serial execution of those transactions.

- ◆ In simple words:

Even if transactions run together (concurrently), the final database state must be as if they ran one by one in some order.

- ◆ Why it is important:

Serializability ensures data consistency and correctness in concurrent transaction processing.

◆ Example:

If two transactions T1 and T2 run together, the schedule is serializable if the result is the same as either:

T1 → T2 (T1 first, then T2), or

T2 → T1 (T2 first, then T1)

✓ If this condition is satisfied → the schedule is serializable

✗ If not → the schedule is not serializable

Concurrency Control

Concurrency control (CC) is a process to ensure that data is updated correctly and appropriately when multiple transactions are concurrently executed in DBMS

Obviously some form of concurrency control mechanism is necessary to enable transactions to run concurrently as far as possible; but controlled in such a way that the effect is the same as if they had been run serially.

